

Kubernetes – Prérequis & Fondamentaux

Jour 1 — Prérequis

Linux, Git et conteneurisation

Kubernetes — Prérequis & Fondamentaux — m2i
Module 1/3

Objectifs du module

À la fin de cette journée, vous serez capable de :

- Naviguer et manipuler un environnement Linux avec les commandes usuelles
- Utiliser Git pour gérer un code source (branches, merge, rebase, conflits)
- Expliquer ce qu'est un conteneur et le différencier d'une VM
- Construire une image Docker optimisée avec un Dockerfile multi-stage
- Orchestrer une stack multi-conteneurs avec Docker Compose

Durée : 7 heures (1 journée)

Plan

1. Commandes Linux essentielles pour Kubernetes
2. Git : rappels et workflow
3. Concepts des conteneurs
4. Images Docker et Dockerfile
5. Réseau et stockage Docker
6. Démo et TP
7. Récap et questions

1. Commandes Linux essentielles

Pourquoi Linux pour Kubernetes ?

“ Kubernetes orchestre des conteneurs Linux. Maîtriser le shell est non négociable. ”

Points essentiels :

- Le control plane et les nœuds tournent sous Linux
- `kubectl exec` ouvre un shell dans un conteneur Linux
- Les manifestes YAML et scripts d'init sont produits/lus en CLI
- Le debug d'un Pod commence presque toujours par des commandes shell

Navigation et manipulation de fichiers

```
cd /etc/kubernetes      # se déplacer
ls -la                  # lister avec attributs
cp -r src/ /backup/     # copier récursif
mv old.yaml new.yaml    # renommer ou déplacer
find . -name "*.yaml"   # rechercher par motif
tree -L 2 manifests/    # arborescence sur 2 niveaux
```

À retenir : `find` est l'outil de référence pour localiser un manifeste perdu dans un repo Helm.

Lecture et édition de fichiers

```
cat /etc/hosts           # afficher
less /var/log/syslog     # parcourir interactivement
head -n 20 kubeconfig    # 20 premières lignes
tail -f /var/log/containers/*.log # suivre en temps réel
vim deployment.yaml      # édition modale
nano configmap.yaml      # édition simple
```

`tail -f` est la commande la plus utilisée en debug runtime.

Redirections, pipes et filtres

```
kubectl get pods | grep CrashLoop          # filtrer
kubectl get pods -o wide | awk '{print $1,$3}' # colonnes ciblées
kubectl describe pod web | sed -n '/Events/, $p' # extraction par motif
cat ports.txt | sort | uniq -c | sort -rn    # top occurrences
wc -l deployment.yaml                      # compter les lignes
```

Pattern : `commande | grep | awk` est la combinaison la plus rentable au quotidien.

Gestion des processus

```
ps -ef | grep kubelet      # processus du nœud
top                        # vue temps réel
kill -TERM 1234            # arrêt propre
kill -9 1234              # arrêt forcé
nohup ./long-job.sh &     # détacher
systemctl status kubelet  # service géré par systemd
journalctl -u kubelet -f  # logs du service
```

`journalctl -u kubelet` est indispensable pour diagnostiquer un nœud qui ne joint pas le cluster.

Permissions et utilisateurs

```
chmod 600 ~/.kube/config      # restreindre l'accès à soi
chown user:group manifest.yaml # propriétaire et groupe
sudo -i                       # session root
id                             # uid, gid, groupes
groups jdoe                    # groupes d'un utilisateur
```

À retenir : `~/.kube/config` doit avoir des droits `600`. Un fichier en `644` peut être lu par d'autres utilisateurs locaux.

Réseau Linux

```
ip a                # interfaces et IPs
ss -tuln            # ports en écoute
netstat -anp | grep 6443 # processus écoutant un port
curl -sk https://10.96.0.1:443 # tester l'API server
wget -O- https://example.com # téléchargement direct
dig kubernetes.default.svc    # résolution DNS
nslookup my-service           # alternative à dig
```

`ss -tuln` remplace `netstat` depuis ~10 ans, plus rapide.

Variables d'environnement et Bash

```
export KUBECONFIG=$HOME/.kube/config
echo "$KUBECONFIG"
kubectl config use-context prod

# Script type
for ns in dev staging prod; do
    kubectl get pods -n "$ns" | tail -n +2 | wc -l
done
```

À retenir : `$KUBECONFIG` peut concaténer plusieurs fichiers avec `:` comme séparateur.

2. Git : rappels et workflow

Le modèle objet de Git

“ Git est une base de données de contenus adressables par hash SHA-1/SHA-256. ”

Quatre objets

- blob (contenu d'un fichier)
- tree (arborescence)
- commit (snapshot + parent)
- tag (référence stable)

Pourquoi c'est important

- Permet le rebase et le cherry-pick
- Explique le merge à 3 voies
- Justifie l'efficacité du diff

Commandes de base

```
git init                # initialiser un repo
git clone git@github.com:m2i/app.git
git status              # état du working tree
git add app.yaml deployment.yaml
git commit -m "feat: add web manifest"
git log --oneline --graph --all
git diff HEAD~1 HEAD -- app.yaml
```

`git log --oneline --graph --all` est la commande la plus efficace pour visualiser un historique.

Gestion des branches

```
git branch                # lister
git branch feature/ingress # créer
git switch feature/ingress # basculer (recommandé depuis 2.23)
git checkout -b hotfix/probe # créer + basculer (legacy)
git merge feature/ingress   # fusionner dans la branche courante
git branch -d feature/ingress # supprimer après merge
```

`git switch` remplace `git checkout` pour le changement de branche.
Utiliser `git restore` pour annuler des changements.

Rebase vs merge

Merge

- Crée un commit de fusion
- Historique non linéaire
- Préserve l'historique des branches
- Utile sur `main` (traçabilité)

Rebase

- Réécrit l'historique
- Historique linéaire et propre
- Force-push si déjà publiée
- Utile en local avant PR

Règle d'or : rebase en local, merge en public.

Résolution de conflit en pratique

```
git switch feature/secrets
git rebase main
# CONFLICT (content): Merge conflict in deployment.yaml
$EDITOR deployment.yaml
# garder les bons blocs, supprimer les marqueurs <<<<<<
git add deployment.yaml
git rebase --continue
```

À retenir : ne jamais committer un fichier contenant encore <<<<<<,
===== ou >>>>>>.

Travail avec un remote

```
git remote -v           # lister les remotes
git push origin feature/ingress # publier une branche
git pull --rebase origin main   # synchro avec rebase
git fetch --all --prune        # rafraîchir sans merger
git tag -a v1.0.0 -m "release"  # tag annoté
git push origin v1.0.0
```

`git pull --rebase` évite les commits de merge parasites sur `main`.

Bonnes pratiques de commit

Format Conventional Commits :

```
<type>(<scope>) : <description>  
  
[corps optionnel expliquant le pourquoi]  
  
[footer optionnel : Refs #123]
```

Types fréquents : feat, fix, chore, refactor, docs, test.

À retenir : un commit = une intention. Si vous écrivez "et", découpez.

3. Concepts des conteneurs

VM vs conteneur

Machine virtuelle

- Émule un matériel complet
- Embarque un OS guest entier
- Démarrage en dizaines de secondes
- Isolation forte (hyperviseur)

Conteneur

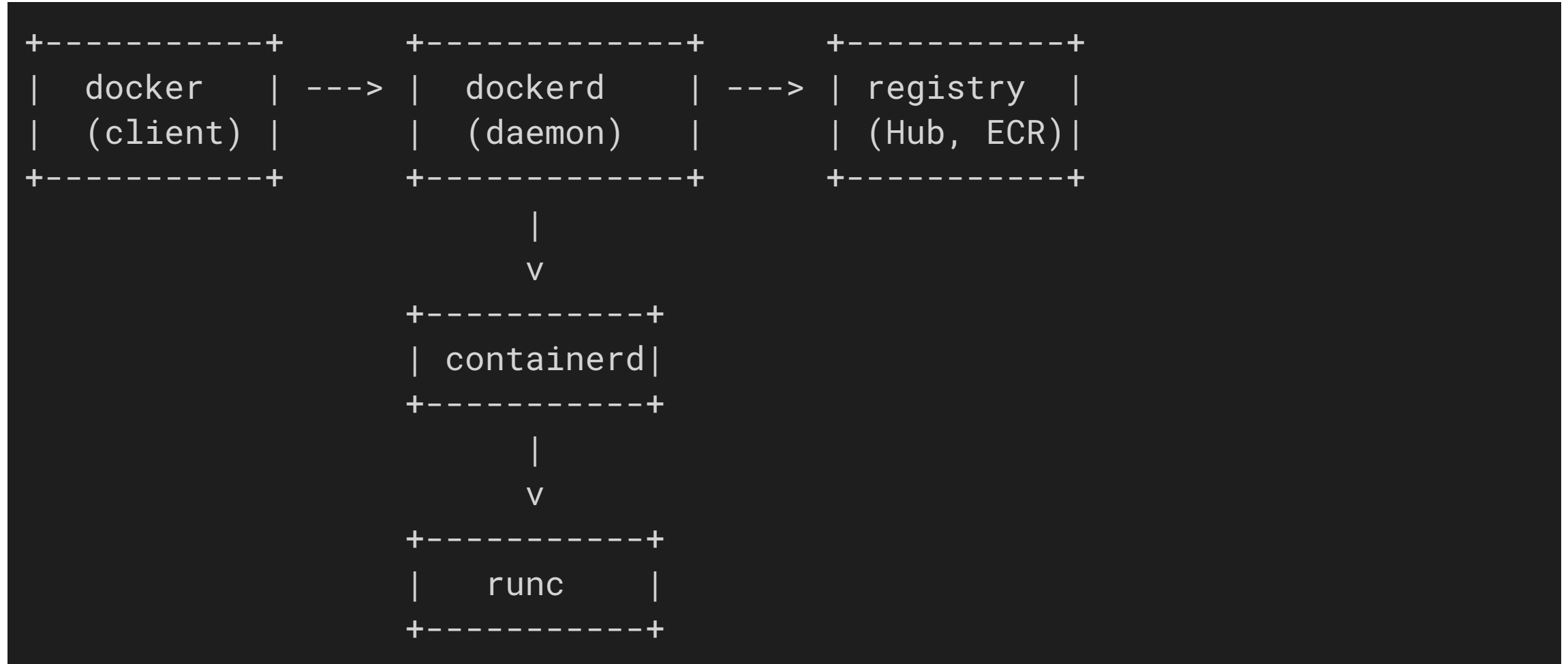
- Partage le noyau de l'hôte
- Embarque seulement les dépendances
- Démarrage en quelques secondes
- Isolation par primitives noyau

Primitives Linux qui rendent les conteneurs possibles

```
# Namespaces : isolation des vues  
lsns -t pid,net,mnt,uts,ipc,user  
  
# Cgroups v2 : limites de ressources  
cat /sys/fs/cgroup/system.slice/docker.service/cpu.max  
  
# UnionFS : empilement de layers  
docker history nginx:1.27
```

Trois piliers : namespaces (isolation), cgroups (limites), UnionFS (layers).

Architecture Docker



Cycle de vie d'un conteneur

```
docker create --name web nginx:1.27    # créé, non démarré
docker start web                        # démarrer
docker pause web                       # geler
docker unpause web                     # reprendre
docker stop web                        # arrêt propre (SIGTERM puis SIGKILL)
docker rm web                          # supprimer
```

Le `docker run` est un raccourci de `create + start + attach`.

4. Images Docker et Dockerfile

Anatomie d'une image

“ Une image est une pile de layers en lecture seule + un manifest JSON. ”

```
docker inspect nginx:1.27 | jq '[0].RootFS'
docker history nginx:1.27           # voir les layers
docker image inspect nginx:1.27 \
  --format='{{.Id}}'                # digest sha256
```

À retenir : tag = nom mutable, digest = identité immuable. En prod, pinner par digest.

Instructions Dockerfile clés

```
FROM eclipse-temurin:21-jre AS runtime
WORKDIR /app
COPY --from=build /workspace/target/app.jar /app/app.jar
ENV JAVA_OPTS="-XX:+UseG1GC"
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar /app/app.jar"]
```

`ENTRYPOINT` définit l'exécutable, `CMD` les arguments par défaut.

Bonnes pratiques d'image

Légèreté

- Images `-slim` ou `-alpine`
- `.dockerignore` strict
- Une seule responsabilité par image

Cache

- Ordonner du moins au plus volatile
- `COPY pom.xml` avant `COPY src/`
- `RUN apt-get update && install` fusionnés

Une image bien faite passe sous 200 Mo pour une app Java.

Multi-stage build

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B -DskipTests package

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /workspace/target/*.jar app.jar
USER 1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Le stage `build` n'est jamais publié : on ne livre que le runtime.

Registry et tags

```
docker tag app:dev registry.m2i.lab/team/app:1.2.3
docker login registry.m2i.lab
docker push registry.m2i.lab/team/app:1.2.3
docker pull registry.m2i.lab/team/app@sha256:abc123
```

Convention de tags :

- `latest` interdit en prod
- Tag immuable : `1.2.3` ou `2026-05-18-a1b2c3`
- Tag de canal : `staging`, `prod` réécrits à chaque release

5. Réseau et stockage Docker

Modes réseau Docker

```
docker network ls
docker network create app-net
docker run -d --network app-net --name db postgres:16
docker run -d --network app-net --name api app:1.0
docker exec api ping db          # résolution DNS interne
```

Quatre modes : `bridge` (défaut), `host` (partage la pile hôte), `none` (isolé), `container:<id>` (partage avec un autre conteneur).

Communication inter-conteneurs

“ Dans un réseau Docker user-defined, le nom de conteneur sert de hostname. ”

```
# docker-compose.yml extrait
services:
  api:
    image: app:1.0
    environment:
      DB_HOST: db          # résolution automatique
  db:
    image: postgres:16
```

À retenir : le DNS Docker user-defined préfigure le DNS Kubernetes (CoreDNS)

Volumes vs bind mounts

Volume Docker

- Géré par Docker
- Portable, sauvegardable
- Cas d'usage : données applicatives

Bind mount

- Chemin hôte arbitraire
- Couplé au système de fichiers hôte
- Cas d'usage : dev local, configs

```
docker volume create pgdata
docker run -v pgdata:/var/lib/postgresql/data postgres:16
docker run -v $(pwd)/conf:/etc/nginx/conf.d:ro nginx:1.27
```

Docker Compose en deux fichiers

```
services:
  api:
    build: ./api
    ports: ["8080:8080"]
    environment:
      DB_HOST: db
      DB_USER: app
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: changeme
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

